

QuizRT — Prezentacja Projektu

QuizRT — Platforma Interaktywnych Quizów Czasu Rzeczywistego

Przedmiot: Zaawansowane Techniki Internetowe

Stack: Spring Boot 3.x · React + Vite · STOMP/SockJS · PostgreSQL 16 · Docker

1. Stos technologiczny

Warstwa	Technologia	Uzasadnienie
Frontend	React 18 + Vite (JavaScript)	SPA, hot-reload, szybki build
Backend	Spring Boot 3.4, Java 21	Spring Data JPA, WebSocket, AOP
Realtime	STOMP over SockJS	pub/sub przez WebSocket z fallbackiem na HTTP
Baza danych	PostgreSQL 16	relacyjny model sesji i wyników
Infrastruktura	Docker + Docker Compose	jeden docker compose up = cały stack

2. Model domenowy (diagram klas)

Encje JPA odzwierciedlają diagram klas z dokumentu architektury 1:1.

Klucze: Long dla treści, UUID dla bytów runtime'owych

```
// Host – właściciel quizów, klucz Long (auto-increment)
@Entity @Table(name = "hosts")
public class Host {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true)
    private String username;
    @Column(name = "password_hash", nullable = false)
    private String passwordHash;
    @OneToMany(mappedBy = "host", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Quiz> quizzes = new ArrayList<>();
}

// GameSession – runtime, klucz UUID (generowany przez PostgreSQL/Hibernate)
@Entity @Table(name = "game_sessions")
public class GameSession {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    @Column(name = "lobby_code", nullable = false, unique = true, length = 6)
    private String lobbyCode;
    @Enumerated(EnumType.STRING)
```

```

private GameState state = GameState.CREATED;
@ManyToOne(fetch = FetchType.LAZY)
private Quiz quiz;
@OneToMany(mappedBy = "gameSession", cascade = CascadeType.ALL)
private List<Player> participants = new ArrayList<>();
private Integer currentQuestionIndex = 0;
}

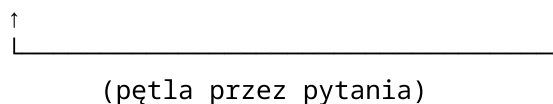
// Player – runtime, klucz UUID
@Entity @Table(name = "players")
public class Player {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    private String nickname;
    @Column(name = "total_score")
    private Integer totalScore = 0;
    @ManyToOne(fetch = FetchType.LAZY)
    private GameSession gameSession;
}

// PlayerAnswer – wynik konkretnej odpowiedzi
@Entity @Table(name = "player_answers")
public class PlayerAnswer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long responseTimeMs; // czas reakcji gracza w ms
    private Boolean correct;
    private Integer pointsAwarded;
    @ManyToOne private Player player;
    @ManyToOne private Question question;
    @ManyToOne private Answer answer;
}

```

3. Maszyna stanów sesji (GameState)

CREATED → LOBBY → QUESTION_ACTIVE → EVALUATING → RANKING_DISPLAY → FINISHED



```

public enum GameState {
    CREATED, LOBBY, QUESTION_ACTIVE, EVALUATING, RANKING_DISPLAY, FINISHED
}

```

Przejścia wymuszane przez GameService: - createSession() → LOBBY - startGame() → QUESTION_ACTIVE + broadcast pierwszego pytania - submitAnswer() (wszyscy odpowiedzieli) lub timeout → EVALUATING → RANKING_DISPLAY - nextQuestion() → QUESTION_ACTIVE lub FINISHED

4. Komunikacja WebSocket — STOMP over SockJS

Konfiguracja brokera

```

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic", "/queue"); // broadcast + prywatne
        registry.setApplicationDestinationPrefixes("/app"); // klient → serwer
        registry.setUserDestinationPrefix("/user");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*")
            .withSockJS(); // fallback dla środowisk bez natywnego WS
    }
}

```

Kanały (diagram komunikacji)

Kanał	Kierunek	Opis
/app/lobby.join	klient → serwer	gracz dołącza do lobby
/app/game.answer	klient → serwer	gracz wysyła odpowiedź
/app/game.nextQuestion	klient → serwer	host przechodzi do kolejnego pytania
/topic/lobby.{sessionId}	serwer → wszyscy	zdarzenia lobby (PLAYER_JOINED, GAME_STARTED)
/topic/game.{sessionId}	serwer → wszyscy	pytania (QuestionEvent) i ranking (RankingEvent)

Handlery STOMP

```

@Controller
public class GameController {

    @MessageMapping("/game.answer")
    public void submitAnswer(@Payload SubmitAnswerRequest req) {
        gameService.submitAnswer(
            req.getSessionId(), req.getPlayerId(),
            req.getQuestionId(), req.getAnswerId(), req.getResponseTimeMs()
        );
    }

    @MessageMapping("/game.nextQuestion")
    public void nextQuestion(@Payload NextQuestionRequest req) {
        gameService.nextQuestion(req.getSessionId(), req.getHostId());
    }
}

@Controller

```

```
public class LobbyController {

    @PostMapping("/lobby.join")
    public void joinLobby(@Payload LobbyJoinRequest req, SimpMessageHeaderAccessor headers) {
        Player player = gameService.joinLobby(req.getLobbyCode(), req.getNickname());
        // zapisanie playerId w sesji WS
        headers.getSessionAttributes().put("playerId", player.getId());
    }
}
```

5. Serwisy (diagram komponentów)

QuizService — CRUD quizów i pytań

Zarządza encjami Host, Quiz, Question, Answer. Rejestracja i logowanie hosta:

```
@Service @Transactional
public class QuizService {

    public HostResponse registerHost(HostRequest req) {
        if (hostRepo.existsByUsername(req.getUsername()))
            throw new IllegalArgumentException("Username already taken");
        Host host = Host.builder()
            .username(req.getUsername())
            .passwordHash(req.getPassword())
            .build();
        return HostResponse.from(hostRepo.save(host));
    }

    public HostResponse loginHost(HostRequest req) {
        Host host = hostRepo.findByUsername(req.getUsername())
            .orElseThrow(() -> new IllegalArgumentException("Invalid credentials"));
        if (!host.getPasswordHash().equals(req.getPassword()))
            throw new IllegalArgumentException("Invalid credentials");
        return HostResponse.from(host);
    }
}
```

GameService — cykl rozgrywki

Kluczowe fragmenty:

Broadcast pytania + zaplanowanie timeout'u po stronie serwera:

```
private void broadcastCurrentQuestion(GameSession session) {
    Question q = session.getQuiz().getQuestions().get(session.getCurrentQuestionIndex());

    QuestionEvent event = QuestionEvent.builder()
        .questionId(q.getId())
        .content(q.getContent())
        .answers(/* lista opcji bez flagi isCorrect */)
        .timeLimitSec(q.resolvedTimeLimitSec())
    }
```

```

        .questionNumber(session.getCurrentQuestionIndex() + 1)
        .totalQuestions(session.getQuiz().getQuestions().size())
        .serverTimestamp(System.currentTimeMillis())
        .build();

broker.convertAndSend("/topic/game." + session.getId(), event);

scheduleTimeout(session.getId(), q.resolvedTimeLimitSec()); // serwer sam przejdzie dalej
    po czasie
}

```

Timeout — wątek schedulera wymaga własnej transakcji:

```

private void handleTimeout(UUID sessionId) {
    // Wątek schedulera nie ma sesji Hibernate – TransactionTemplate tworzy nową transakcję
    txTemplate.executeWithoutResult(status ->
        sessionRepo.findById(sessionId).ifPresent(session -> {
            if (session.getState() == GameState.QUESTION_ACTIVE) {
                evaluateRound(session);
            }
        })
    );
}

```

Early-exit — zakończenie rundy gdy wszyscy odpowiedzą przed timeoutem:

```

public void submitAnswer(UUID sessionId, UUID playerId, Long questionId, ...) {
    // ... zapis odpowiedzi, obliczenie punktów ...

    long totalPlayers = session.getParticipants().size();
    long answeredCount = playerAnswerRepo.findByPlayerGameSessionId(sessionId).stream()
        .filter(a -> a.getQuestion().getId().equals(questionId))
        .count();

    if (answeredCount >= totalPlayers) {
        cancelTimeout(sessionId); // anuluj zaplanowany timeout
        evaluateRound(session);   // przejdź od razu do rankingu
    }
}

```

Zarządzanie timeoutami — mapa Future per sesja:

```

private final Map<UUID, ScheduledFuture<?>> timeoutFutures = new HashMap<>();

private void scheduleTimeout(UUID sessionId, int timeLimitSec) {
    cancelTimeout(sessionId); // anuluj poprzedni jeśli istnieje
    ScheduledFuture<?> future = taskScheduler.schedule(
        () -> handleTimeout(sessionId),
        Instant.now().plusSeconds(timeLimitSec)
    );
    timeoutFutures.put(sessionId, future);
}

private void cancelTimeout(UUID sessionId) {
    ScheduledFuture<?> future = timeoutFutures.remove(sessionId);
}

```

```

    if (future != null) future.cancel(false);
}

```

RankingService — punktacja i broadcast rankingu

```

@Service
public class RankingService {

    /**
     * Wzór Kahoot:  $1000 \times (1 - \text{czas\_odpowiedzi} / \text{limit\_czasu})$ 
     * Odpowiedź błędna lub po czasie → 0 pkt
     */
    public int calculateScore(boolean correct, long responseTimeMs, int timeLimitSec) {
        if (!correct) return 0;
        long timeLimitMs = (long) timeLimitSec * 1000;
        if (responseTimeMs >= timeLimitMs) return 0;
        double ratio = (double) responseTimeMs / timeLimitMs;
        return Math.max(0, (int) Math.round(1000 * (1.0 - ratio)));
    }

    public void broadcastRanking(UUID sessionId, int currentQ, int totalQ, boolean
        gameFinished) {
        List<PlayerScore> ranking = getLiveRanking(sessionId); // sortowane po totalScore
        DESC
        RankingEvent event = RankingEvent.builder()
            .ranking(ranking)
            .gameFinished(gameFinished)
            .currentQuestion(currentQ)
            .totalQuestions(totalQ)
            .build();
        broker.convertAndSend("/topic/game." + sessionId, event);
    }
}

```

Przykłady punktacji przy limicie 10 sekund:

Czas odpowiedzi	Obliczenie	Punkty
1s	$1000 \times (1 - 0.1)$	900
5s	$1000 \times (1 - 0.5)$	500
9s	$1000 \times (1 - 0.9)$	100
błędna / po czasie —		0

6. AOP — AuditAspect

Przekrojowe logowanie wszystkich wywołań warstwy serwisów. Pointcut obejmuje pakiet service.*.

```

@Aspect @Component @Slf4j
public class AuditAspect {

    @Pointcut("execution(* pl.agh.zti.quiz.service.*.*(..))")
    public void serviceLayer() {}
}

```

```

@Around("serviceLayer()")
public Object audit(ProceedingJoinPoint pjp) throws Throwable {
    String method = pjp.getSignature().toShortString();
    long start = System.currentTimeMillis();

    log.info("[AUDIT] >> {} args={}", method, Arrays.toString(pjp.getArgs()));
    try {
        Object result = pjp.proceed();
        log.info("[AUDIT] << {} OK ({}ms)", method, System.currentTimeMillis() - start);
        return result;
    } catch (Throwable ex) {
        log.warn("[AUDIT] << {} EXCEPTION {} ({}ms)", method, ex.getMessage(),
            System.currentTimeMillis() - start);
        throw ex;
    }
}

```

Logi w czasie gry (fragment):

```

[AUDIT] >> GameService.submitAnswer(..) args=[abc-123, uuid-p1, 3, 10, 1240]
[AUDIT] << GameService.submitAnswer(..) OK (12ms)
[AUDIT] >> RankingService.broadcastRanking(..) args=[abc-123, 1, 3, false]
[AUDIT] << RankingService.broadcastRanking(..) OK (5ms)

```

7. REST API

Metoda	Ścieżka	Opis
POST	/api/hosts/register	rejestracja hosta
POST	/api/hosts/login	logowanie hosta
GET	/api/hosts/{id}	dane hosta
GET	/api/quizzes?hostId=	lista quizów hosta
POST	/api/quizzes?hostId=	utwórz quiz
GET	/api/quizzes/{id}	szczegóły quizu
POST	/api/quizzes/{id}/questions	dodaj pytanie
POST	/api/questions/{id}/answers	dodaj odpowiedź
POST	/api/sessions?quizId=&hostId=	utwórz sesję gry
POST	/api/sessions/{id}/start?hostId=	rozpocznij grę
GET	/api/sessions/by-code?code=	znajdź sesję po kodzie
GET	/api/sessions/{id}/current-question	aktywne pytanie (lub 204)

Obsługa błędów przez @RestControllerAdvice:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, String>> handleIllegal(IllegalArgumentException ex) {
        return ResponseEntity.badRequest().body(Map.of("error", ex.getMessage()));
    }
}

```

```
}  
}
```

8. Frontend — kluczowe rozwiązania

Połączenie WebSocket (SockJS + STOMP)

```
// api/ws.js  
import { Client } from '@stomp/stompjs'  
import SockJS from 'sockjs-client'  
  
export function connectWS(onConnected) {  
  const c = new Client({  
    websocketFactory: () => new SockJS('/ws'),  
    reconnectDelay: 0,  
    onConnect: () => onConnected(c),  
  })  
  c.activate()  
  return c  
}
```

WaitingRoom — prawidłowe zapisywanie playerId

Kluczowy bug: PLAYER_JOINED to broadcast do wszystkich graczy. Bez filtrowania każdy gracz nadpisywałby swój playerId identyfikatorem ostatnio dołączonej osoby.

```
// WaitingRoom.jsx – zapisuj playerId TYLKO dla siebie  
if (event.type === 'PLAYER_JOINED') {  
  setPlayers(event.participants ?? [])  
  if (event.playerId && event.nickname === nickname) { // ← filtr po własnym nickname  
    localStorage.setItem('playerId', event.playerId)  
  }  
}
```

PlayerGame — recovery aktywnego pytania

Gracz może dołączyć gdy pytanie jest już aktywne (np. chwilowe rozłączenie). Endpoint HTTP zwraca aktualny stan:

```
// Po subskrypcji WS – pobierz pytanie które mogło być wysłane przed połączeniem  
http.get(`/sessions/${sessionId}/current-question`)  
  .then(r => {  
    if (r.status === 200 && r.data?.questionId) {  
      setQuestion(r.data)  
      setPhase('QUESTION')  
      questionStartRef.current = Date.now()  
    }  
  })  
  .catch(() => {}) // 204 No Content = brak aktywnego pytania
```

Mierzenie czasu odpowiedzi po stronie klienta


```
// Zapamiętaj timestamp odebrania pytania
questionStartRef.current = Date.now()

// Przy wysyłaniu odpowiedzi oblicz czas reakcji
const responseTimeMs = Date.now() - (questionStartRef.current ?? Date.now())

client.publish({
  destination: '/app/game.answer',
  body: JSON.stringify({ sessionId, playerId, questionId, answerId, responseTimeMs }),
})
```

Timer z wizualną zmianą koloru

```
// Timer.jsx – kolor zmienia się wraz z upływem czasu
const color = left > seconds * 0.5 ? '#4caf50' // zielony > 50%
              : left > seconds * 0.25 ? '#ff9800' // pomarańczowy > 25%
              : '#f44336' // czerwony ≤ 25%
```

QR kod do dołączenia (react-qr-code)

```
// HostGame.jsx – w lobby host widzi QR prowadzący do strony gracza
import { QRCode } from 'react-qr-code' // named export, nie default

<QRCode
  value={`${window.location.origin}/player?code=${session.lobbyCode}`}
  size={160}
/>
```

Gracz skanuje → JoinLobby.jsx odczytuje kod z URL:

```
// JoinLobby.jsx
const [searchParams] = useSearchParams()
const [code, setCode] = useState(searchParams.get('code') ?? '')
// pole kodu jest już wypełnione – gracz wpisuje tylko nick i dołącza
```

9. Infrastruktura Docker

Wieloetapowy build backendu (Maven → JRE Alpine)

```
# backend/quiz/Dockerfile
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q # cache warstwy z zależnościami
COPY src ./src
RUN mvn package -DskipTests -q

FROM eclipse-temurin:21-jre-alpine # ~200MB zamiast ~800MB
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Wieloetapowy build frontendu (Node → nginx)

```
# frontend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # vite build → dist/

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

nginx jako reverse proxy

```
# frontend/nginx.conf
server {
    listen 80;
    root /usr/share/nginx/html;

    location / {
        try_files $uri $uri/ /index.html; # SPA fallback
    }

    location /api/ {
        proxy_pass http://backend:8080/api/;
    }

    location /ws {
        proxy_pass http://backend:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade"; # WebSocket upgrade
        proxy_read_timeout 3600s;
    }
}
```

docker-compose — jeden komend uruchamia wszystko

```
services:
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U quiz -d quiz"]

  backend:
    build: { context: backend/quiz }
    depends_on:
      db: { condition: service_healthy } # czeka na gotowość DB
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/quiz
```

```
frontend:
  build: { context: frontend }
  depends_on: [backend]
  ports: ["80:80"]
```

```
docker compose up -d          # start całego stacku
docker compose up -d --build  # rebuild po zmianach w kodzie
```